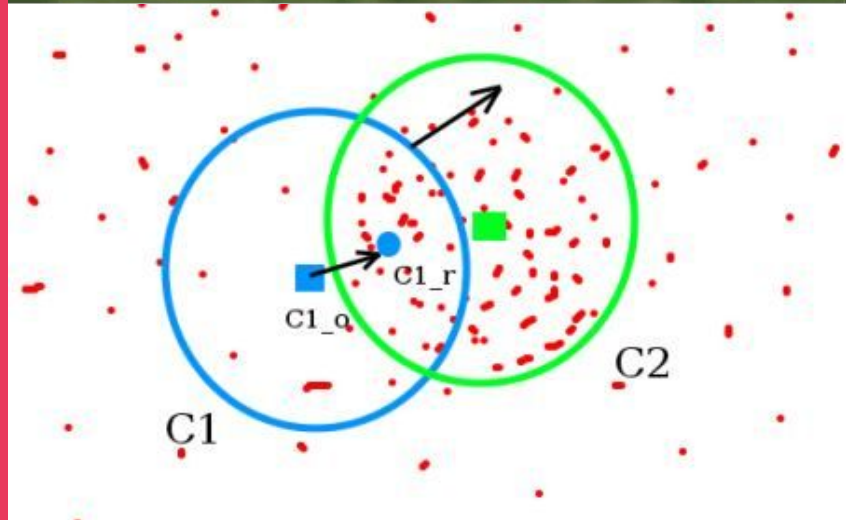
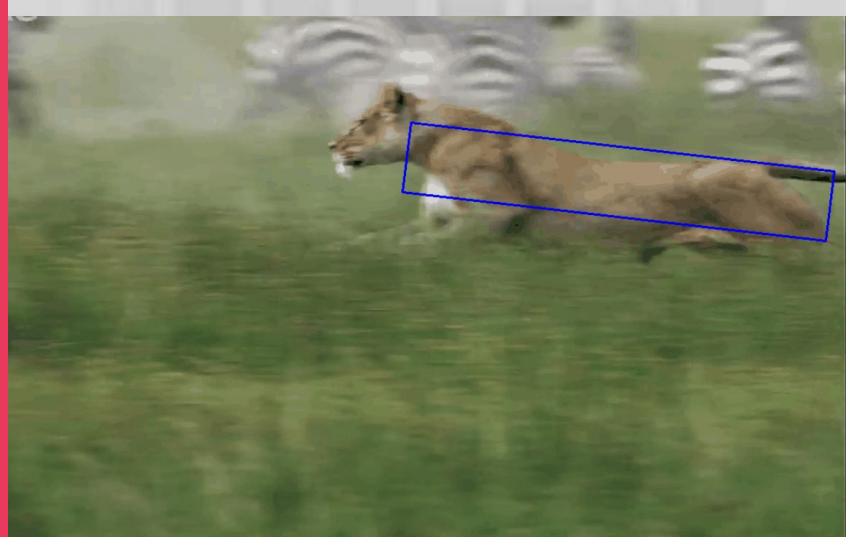


OBJECT TRACKING

CS 663 COURSE PROJECT

Jigyasa Chouhan (23b1083)

Raghav Goyal (23b0908)



INTRODUCTION

Meanshift and Camshift algorithms

When we look at an image, we see a bunch of pixels. All of these pixels represent color values, and if we consider an image as a color map, we can follow that color pattern. That is exactly what MeanShift and CamShift do.

MeanShift algorithm uses a simple trick to find the centre of the object being tracked in every frame using the position of the object in the previous frame of the video.

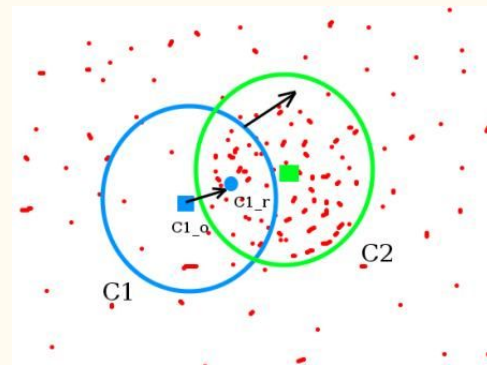
CamShift algorithm is built on top of mean shift by adding a couple of more features (surprise) to enhance the object tracking experience in a video.

STEP I: MEAN SHIFT

$$x_c = \frac{m_{10}}{m_{00}}, \quad y_c = \frac{m_{01}}{m_{00}},$$

- 1) We first remember the hue distribution of the original object marked by the user.
- 2) At the next frame, we would expect it to have moved a bit. We associate with each pixel the probability of it being part of the object (by looking up its probability in the histogram).
- 3) Now we use the mean shift algorithm to correct the position. A brief-ish description of the intuition and working of mean shift is:

Consider you have a set of points. (It can be a pixel distribution like histogram backprojection). You are given a small window (may be a circle) and you have to move that window to the area of maximum pixel density (or maximum number of points). It is illustrated in the simple image shown. The initial window is shown in blue circle with the name "C1". Its original center is marked in blue rectangle, named "C1_o". But if you find the centroid of the points inside that window, you will get the point "C1_r" (marked in small blue circle) which is the real centroid of the window. Surely they don't match. So move your window such that the circle of the new window matches with the previous centroid. Again find the new centroid. Most probably, it won't match. So move it again, and continue the iterations such that the center of window and its centroid falls on the same location (or within a small desired error). So finally what you obtain is a window with maximum pixel distribution.



STEP II: WINDOW RESIZING

Consider moments of the pixels being calculated using the aforementioned probability weights as

$$M_{pq} = \sum_x \sum_y x^p y^q w(x, y)$$

- Scale the window by a factor:

$$S = \sqrt{M_{00}}$$

Why this works is because we expect the sum of probabilities of it being in the image as a measure of the proportion of the portion of the ROI covered by the object.

The aspect ratio is maintained with respect to the original target given.

STEP III: ANGLE -> CAMSHIFT

Want to detect tilting objects?
Quite some math!

$$\mu_{20} = \frac{m_{20}}{m_{00}} - x_c^2, \quad \mu_{02} = \frac{m_{02}}{m_{00}} - y_c^2, \quad \mu_{11} = \frac{m_{11}}{m_{00}} - x_c y_c.$$

$$C = \begin{bmatrix} \mu_{20} & \mu_{11} \\ \mu_{11} & \mu_{02} \end{bmatrix}$$

$$\theta = \frac{1}{2} \tan^{-1} \left(\frac{2\mu_{11}}{\mu_{20} - \mu_{02}} \right) a = \mu_{20} + \mu_{02},$$

STEP III: ANGLE -> CAMSHIFT

$$a = \mu_{20} + \mu_{02}, \quad b = \sqrt{4\mu_{11}^2 + (\mu_{20} - \mu_{02})^2}$$

$$\lambda_1 = \frac{a + b}{2}, \quad \lambda_2 = \frac{a - b}{2}$$

$$\text{major} = 4\sqrt{|\lambda_1|}, \quad \text{minor} = 4\sqrt{|\lambda_2|}$$

You've got the ROI required!

STEP IV: OPENCV CAMSHIFT

Open CV has its own implementation of the camShift algorithm. Let's compare how close we have come to it with our novice knowledge!

TIME TO BATTLE!

References

- <https://www.cse.psu.edu/~rtc12/CSE598G/introMeanShift.pdf>
- <https://medium.com/@claudio.vindimian/understanding-and-implementing-the-camshift-object-tracking-algorithm-python-81587c24eda8>
- [Real-Time Tracking of Non-Rigid Objects using Mean Shift](#) : Dorin Comaniciu, Visvanathan Ramesh, Peter Meer
- [Real-time Non-rigid Object Tracking Using CAMShift with Weighted Back Projection](#) : Lei Sun, Bingrong Wang, Takeshi Ikenaga



**THANK
YOU!**